

Actividad 4

Alumno

Alan Lam Páez

Matrícula

T03063809

Materia

Gestión y Automatización de
Pruebas de Software

Maestro(a)

Cristopher Gerardo Gaytán Díaz

Correo

al03063809@tecmilenio.mx

Fecha

18/09/2026

Seguridad y principios DevSecOps

Pruebas automatizadas en CI/CD y mejora de la calidad

La implementación de pruebas automatizadas permite verificar de manera repetible que los servicios de una API continúen funcionando correctamente después de realizar cambios. En este proyecto se utilizaron pruebas desarrolladas en Postman y ejecutadas mediante Newman para validar diferentes métodos HTTP, incluyendo GET, POST, PUT y DELETE.

Las pruebas comprueban aspectos como los códigos de estado HTTP, la existencia de propiedades esperadas en las respuestas JSON y la presencia de determinados encabezados. Esto permite detectar cambios inesperados en el comportamiento de los servicios sin depender únicamente de una revisión manual.

Posteriormente, la colección de Postman fue integrada en un pipeline de GitHub Actions. De esta manera, las pruebas pueden ejecutarse automáticamente cuando se realizan cambios en el repositorio. Si alguna prueba o assertion falla, Newman reporta el error y el workflow de GitHub Actions se marca como fallido. Esto permite identificar problemas de manera temprana y evita que cambios que afecten el funcionamiento de la API pasen desapercibidos.

Autenticación mediante tokens y seguridad de APIs

La autenticación mediante tokens permite controlar el acceso a determinados recursos de una API sin tener que enviar las credenciales del usuario en cada solicitud. En este proyecto se implementó un flujo de autenticación utilizando un endpoint de login que genera un accessToken.

El token obtenido durante el proceso de login se almacena como una variable de entorno de Postman y posteriormente se utiliza mediante el esquema Bearer Token para realizar una solicitud autenticada al endpoint de información del usuario.

Este mecanismo contribuye a la seguridad porque permite que la API verifique que una solicitud proviene de un usuario o cliente que previamente se autenticó. Además, el uso de tokens con una duración limitada puede reducir el impacto de una credencial expuesta. En un entorno real, estos tokens y otros secretos deben administrarse mediante mecanismos seguros de almacenamiento y nunca incluirse directamente en el código fuente o en repositorios públicos.

Riesgos comunes detectados mediante pruebas automatizadas

Las pruebas automatizadas pueden ayudar a identificar diferentes problemas relacionados con la disponibilidad, funcionamiento y comportamiento esperado de una API. Entre ellos se encuentran respuestas con códigos HTTP inesperados, propiedades faltantes en los cuerpos de respuesta, cambios en el formato de los datos y problemas relacionados con la autenticación.

En este proyecto, por ejemplo, se comprobó que una solicitud autenticada recibiera un código 200 OK y que la respuesta contuviera información esperada como el identificador, nombre de usuario y correo electrónico. También se implementó un escenario de Mock Server que devuelve una respuesta 404 Not Found, permitiendo comprobar cómo se comportan las pruebas ante un recurso inexistente.

Aunque estas pruebas no sustituyen una auditoría de seguridad completa, permiten detectar automáticamente ciertos errores antes de que lleguen a etapas posteriores del desarrollo.

Integración de DevSecOps en el pipeline

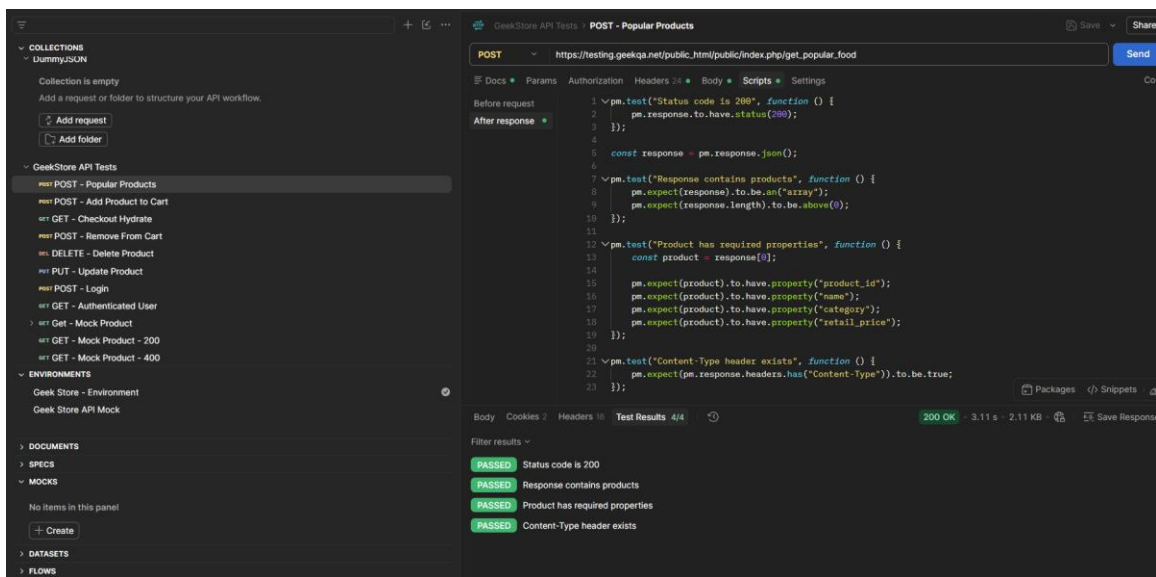
La filosofía DevSecOps busca incorporar la seguridad y las pruebas durante todo el ciclo de desarrollo, en lugar de considerarlas una actividad independiente realizada únicamente al final del proyecto.

En este proyecto, esta filosofía se aplica mediante la integración de las pruebas de API en el proceso de CI/CD. La colección desarrollada en Postman puede ser ejecutada mediante Newman desde la línea de comandos, y

posteriormente GitHub Actions automatiza esta ejecución dentro del repositorio.

El pipeline instala las herramientas necesarias, ejecuta la colección de Postman y verifica las assertions definidas. Como resultado, los cambios realizados en el proyecto pueden ser validados automáticamente. Esta integración permite incorporar controles de calidad y algunas validaciones de seguridad directamente en el proceso de desarrollo, reduciendo la posibilidad de que errores sean detectados únicamente después de la implementación.

Evidencias



- Ejemplo de pruebas realizadas al seleccionar artículos en GeekStore

GeekStore API Tests > GET - Authenticated User

GET https://dummyjson.com/auth/me

Auth type: Bearer Token

Token: accessToken

Body: Cookies: Headers: Test Results: 5/5

200 OK · 120 ms · 1.81 KB

Filter results

- PASSED Status code is 200
- PASSED Authenticated user has an ID
- PASSED Authenticated user has a username
- PASSED Authenticated user has an email
- PASSED Content-Type is JSON

GeekStore API Tests

Collection is empty

Add a request or folder to structure your API workflow.

Add request

Add folder

GeekStore API Tests

- POST - Popular Products
- POST - Add Product to Cart
- GET - Checkout Hydrate
- POST - Remove From Cart
- DELETE - Delete Product
- PUT - Update Product
- POST - Login
- GET - Authenticated User
- GET - Mock Product
- GET - Mock Product - 200
- GET - Mock Product - 400

ENVIRONMENTS

- Geek Store - Environment
- Geek Store API Mock

DOCUMENTS

SPECS

MOCKS

No items in this panel

Create

DATASETS

FLAWS

```

PS C:\Hiertan\Academic\Tecmilenio\Semestre7\GestionAutoPruebasSoft\actividades\act4\Postman\GeekStore>
newman run "Geek Store API Tests.postman_collection.json" -e "Geek Store - Environment.postman_environment.json"
newman

GeekStore API Tests

→ POST - Popular Products
  POST https://testing.geekqa.net/public_html/public/index.php/get_popular_food [200 OK, 3.06kB, 719ms]
  ✓ Status code is 200
  ✓ Response contains products
  ✓ Product has required properties
  ✓ Content-Type header exists

→ POST - Add Product to Cart
  POST https://testing.geekqa.net/public_html/public/index.php/productModifiers [200 OK, 1.59kB, 229ms]
  ✓ Status code is 200
  ✓ Response contains product_info
  ✓ Product info has required properties
  ✓ Content-Type header exists

→ GET - Checkout Hydrate
  GET https://pci-connect.squareupsandbox.com/payments/hydrate?applicationId=sandbox-sq0ldb-mBilTrRZiJ
  Vkw56iBuwYUg&hostname=testing.geekqa.net&locationId=LGy7FG0N2Z9FW&version=1.84.5 [200 OK, 2.18kB, 451ms]
  ✓ Status code is 200
  ✓ Response contains sessionId
  ✓ Response contains instanceId
  ✓ Response contains paymentMethodSupportDetail
  ✓ Response contains merchantDetails
  ✓ Content-Type header exists

→ POST - Remove From Cart
  POST https://testing.geekqa.net/public_html/public/index.php/removeCart [200 OK, 1.32kB, 225ms]
  ✓ Status code is 200
  ✓ Response confirms product was removed
  ✓ Content-Type header exists

→ DELETE - Delete Product
  DELETE https://dummyjson.com/products/1 [200 OK, 2.55kB, 117ms]
  ✓ Status code is 200
  ✓ Response confirms product was deleted
  ✓ Response contains deletedOn
  ✓ Content-Type is JSON

→ PUT - Update Product
  PUT https://dummyjson.com/products/1 [200 OK, 1.53kB, 76ms]
  ✓ Status code is 200

```

- Pruebas automatizadas CI/CD (en base a las pruebas en scripts por request)

```
Windows PowerShell x Windows PowerShell x + v
✓ Product price was updated
✓ Response contains product ID
✓ Content-Type is JSON

→ POST - Login
POST https://dummyjson.com/auth/login [200 OK, 2.84kB, 67ms]
✓ Status code is 200
✓ Response contains access token
✓ Response contains refresh token
✓ Response contains username
✓ Access token was saved

→ GET - Authenticated User
GET https://dummyjson.com/auth/me [200 OK, 2.41kB, 62ms]
✓ Status code is 200
✓ Authenticated user has an ID
✓ Authenticated user has a username
✓ Authenticated user has an email
✓ Content-Type is JSON

→ Get - Mock Product
GET https://dummyjson.com/products/1 [200 OK, 2.52kB, 38ms]

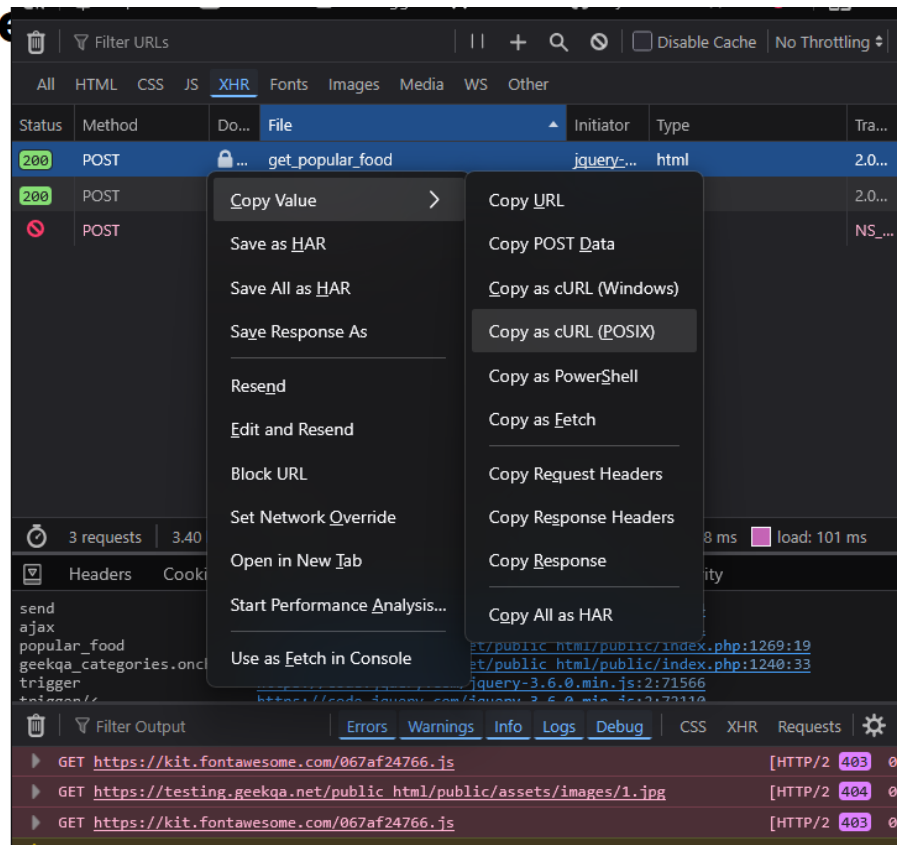
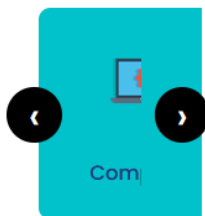
→ GET - Mock Product - 200
GET https://fc508bd5-bcd9-4c46-a48e-ec1e8fc5c1e2.mock.pstmn.io/products/1 [200 OK, 3.26kB, 387ms]

→ GET - Mock Product - 400
GET https://fc508bd5-bcd9-4c46-a48e-ec1e8fc5c1e2.mock.pstmn.io/products/999999 [404 Not Found, 1.47kB, 258ms]
✓ Status code is 404
✓ Response contains error message
✓ Content-Type is JSON
```

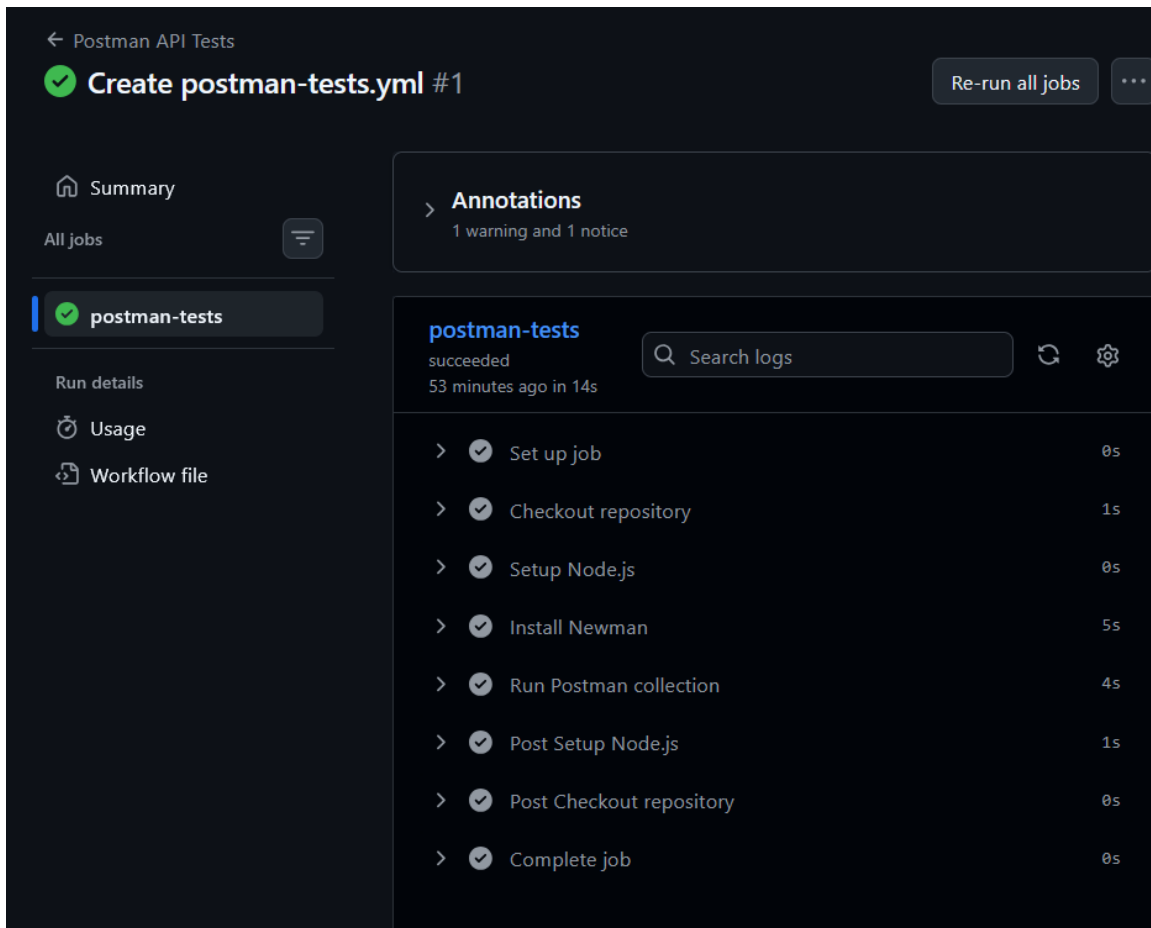
	executed	failed
iterations	1	0
requests	11	0
test-scripts	9	0
prerequest-scripts	0	0
assertions	39	0
total run duration: 3.4s		
total data received: 10.79kB (approx)		
average response time: 239ms [min: 38ms, max: 719ms, s.d.: 200ms]		

favourite product

1. Sign up or login
2. Add your products to the cart
3. Select wrapping
4. Proceed to checkout
5. Make a payment



- Se utilizo cURL (Windows) para importar los endpoints en Postman.



- Ejecución de pruebas en Github Actions

Retrospectiva del proyecto

- **¿Qué avances concretos se han logrado desde la retrospectiva anterior?** Desde la retrospectiva anterior se completó la configuración de las pruebas de API en Postman, incluyendo solicitudes GET, POST, PUT y DELETE. También se implementó autenticación mediante tokens, respuestas simuladas con Mock Server y la ejecución automatizada de las pruebas mediante Newman y GitHub Actions.
- **¿Qué impedimentos se encontraron y cómo se solucionaron?** Se encontraron dificultades debido a que Geek Store no contaba con todos los métodos HTTP requeridos y algunas solicitudes presentaban

protección CSRF. Para solucionarlo, se utilizaron servicios externos como DummyJSON para complementar los métodos faltantes y se importaron las solicitudes reales de Geek Store mediante cURL. También se corrigió la configuración del token y la exportación de la colección para que Newman pudiera ejecutarla correctamente.

- **¿Qué tareas siguen pendientes en el prototipo funcional?** El prototipo funcional cumple con los requisitos principales de la actividad. Como tareas pendientes quedan ampliar la cobertura de pruebas, agregar más escenarios de error y mejorar los reportes generados por el pipeline de CI/CD.